

자기소개서

백엔드 개발 직무 지원 | Spring Boot · Java · Redis · 대규모 트래픽 처리

1. 지원 동기 및 회사에서 이루고 싶은 일

저는 기능을 구현하는 데 그치지 않고, 그 기능이 트래픽과 동시성 앞에서도 안정적으로 동작하는지를 먼저 고민하는 백엔드 개발자입니다. 많은 사용자가 동시에 접속하고 데이터가 빠르게 누적되는 환경에서 응답 성능과 데이터 정합성을 함께 지켜내는 일에 깊은 흥미를 느낍니다. 대규모 트래픽과 안정성을 동시에 요구하는 귀사의 서비스 환경이야말로 제 역량을 가장 잘 발휘하고 또 성장시킬 수 있는 무대라고 생각하여 지원하게 되었습니다.

소셜 기록 플랫폼 Deokive를 개발하며 저는 단순한 CRUD 구현을 넘어 인프라 레벨의 설계를 직접 담당했습니다. 데이터가 누적될수록 느려지는 갤러리와 피드 조회의 딜레이를 해결하기 위해, 정렬 조건에 부합하는 ID 목록을 Redis에 캐싱하고 이를 기반으로 조회하는 캐싱 아키텍처를 설계했으며, 여기에 커버링 인덱스 전략을 결합하여 불필요한 폴스캔을 제거했습니다. 요청이 집중되는 좋아요 기능에서는 Redis Lua 스크립트의 원자적 연산으로 즉시 응답하고 실제 데이터베이스 반영은 RabbitMQ로 비동기 처리함으로써, 실시간 응답성과 데이터베이스 부하라는 상충하는 두 목표를 동시에 잡았습니다. 이 과정에서 '더 빠른 길'이 아니라 '더 견고한 길'을 찾는 사고를 체득했습니다.

입사 후에는 인증·인가 레이어와 대규모 동시 요청 처리 구조를 더 안정적으로 개선하는 데 기여하고 싶습니다. 나아가 캐싱과 메시지 큐, 비동기 이벤트 기반 설계 경험을 살려 서비스가 더 적은 자원으로 더 많은 트래픽을 안정적으로 감당하도록 만드는 백엔드 엔지니어로 성장하고 싶습니다.

2. 성장 과정

저의 성장에 가장 큰 전환점은 Deokive 프로젝트에서 백엔드 설계를 주도적으로 책임지게 된 경험입니다. 백엔드 코드의 대부분을 직접 작성하며 처음에는 동작하는 기능을 만드는 데 집중했지만, 데이터가 쌓이고 조회 요청이 몰리자 잘 동작하던 기능들이 눈에 띄게 느려지는 것을 보며 '코드가 작동하는 것'과 '서비스가 안정적으로 운영되는 것'은 전혀 다른 문제임을 깨달았습니다.

그때부터 저는 운영 환경을 의식한 설계를 습관으로 삼았습니다. 빈번한 쓰기가 발생하는 조회수는 데이터베이스에 직접 반영하지 않고 Redis에 우선 기록한 뒤 스케줄러로 일괄 반영하는 Write-Back 전략을 도입하여 쓰기 부하와 커넥션 풀 고갈 위험을 함께 낮췄고, 외부 인프라 장애가 전체 배치를 중단시키지 않도록 재시도와 결함 허용(Fault-Tolerant) 설정을 더했습니다. 화려한 기술을 먼저 떠올리기보다 문제의 본질에 맞는 가장 단순하고 견고한 해법을 찾는 태도가 이 시기에 자리 잡았습니다.

특히 '작동하는 설계를 먼저 만들고, 측정하여 개선한다'는 원칙을 세운 것이 가장 큰 성장 동력이었습니다. 캐시를 도입하면 당연히 빨라질 것이라는 막연한 기대 대신 캐시 적중과 무효화 시점을 직접 따져보며 정합성과 성능이 함께 유지되도록 다듬었습니다. 실패를 두려워하지 않고 근거를 가지고 개선해 나가는 자세가 지금의 저를 만든 가장 큰 동력입니다.

3. 다른 사람에게 꼭 전하고 싶은 경험

첫째, 분산 환경을 전제로 설계해야 한다는 것입니다. 동일한 아카이브에 같은 날짜의 스티커가 중복 등록되는 문제를 마주했을 때 처음에는 애플리케이션 레벨의 락을 떠올렸습니다. 그러나 서버가 여러 인스턴스로 확장되면 애플리케이션 락은 무력화된다는 점을 인지하고, 데이터베이스의 고유 제약조건(Unique Constraint)을 활용해 중복을 원천 차단했습니다. 자신의 코드가 보장하는 것과 보장하지 못하는 것을 솔직하게 구분하고 단일 서버의 한계를 먼저 인정하는 태도가 더 견고한 설계로 이어진다는 것을 배웠습니다.

둘째, 만든 기능을 끝까지 책임져야 한다는 것입니다. 실시간 알림을 SSE(Server-Sent Events)로 구현한 뒤 기능 자체는 잘 동작했지만, 네트워크가 끊긴 커넥션이 정리되지 않아 서버 자원이 조용히 누수되는 문제를 발견했습니다. 저는 연결을 일괄 관리하는 레지스트리를 두고 주기적인 하트비트(Ping)로 끊긴 연결을 즉시 감지하고 해제하도록 보완하여, 장시간 운영에서도 메모리가 안정적으로 유지되도록 만들었습니다. 기능을 '구현 완료'하는 것과 '운영 가능한 상태로 마무리'하는 것은 다르며, 눈에 보이지 않는 자원까지 챙기는 것이 백엔드 개발자의 책임임을 전하고 싶습니다.